GitForesight: Especificação de Casos de Uso

Versão 1.0

Histórico de Revisão

Versão	Data	Autor	Descrição
1.0	09/04/2026	Maria Fernanda L. R. de Sousa	Especificação dos casos de uso de acordo com o Documento de Visão.

Visão Geral dos Atores

- **Líder Técnico / Gestor de Projeto:** Responsável pela configuração institucional, integração de ferramentas e monitoramento de métricas estratégicas de risco e retenção de conhecimento.
- **Desenvolvedor:** Engenheiro de software responsável pelo ciclo de vida local do código, criação de commits e execução de rotinas de merge.
- **Sistema (GitForesight):** Mecanismo automatizado responsável por interceptações, verificações estáticas, aplicação de heurísticas e orquestração de fluxos pós-evento.

Casos de Uso: Integração e Gestão Estratégica

UC01 - Autenticar e Conectar Repositório

- **Identificador:** UC01
- **Ator Principal:** Líder Técnico / Gestor de Projeto
- **Objetivo:** Estabelecer canal seguro de comunicação entre a plataforma e o provedor Git remoto.
- **Pré-condições:** O usuário deve possuir credenciais ativas e privilégios administrativos no provedor Git.
- **Fluxo Principal:**
    a. O usuário solicita a conexão com uma conta do GitHub ou GitLab.
    b. O sistema redireciona a requisição via protocolo OAuth 2.0 solicitando escopo exclusivo de leitura (*read-only*).
    c. O usuário autoriza o acesso no provedor externo.
    d. O sistema recebe a resposta de autorização e armazena o token de acesso de forma criptografada na base de dados.

    e. O sistema recupera e exibe a listagem de repositórios disponíveis para vinculação.
- **Pós-condições:** Repositórios sincronizados e prontos para inspeção contínua.

UC02 - Simular Cenário de Perda de Conhecimento

- **Identificador:** UC02
- **Ator Principal:** Líder Técnico
- **Objetivo:** Projetar impactos arquiteturais e gargalos operacionais decorrentes do desligamento ou remanejamento de colaboradores-chave.
- **Pré-condições:** O repositório deve ter seu histórico previamente analisado com a matriz de titularidade de código (*Code Ownership*) calculada.
- **Fluxo Principal:**
    a. O líder técnico acessa o módulo analítico e seleciona um ou mais colaboradores para simular ausência.
    b. O sistema recalcula dinamicamente a matriz de dependência e propriedade de código sem os registros dos colaboradores selecionados.
    c. O sistema avalia o impacto e gera novos níveis de criticidade e risco para os módulos afetados.
    d. O sistema exibe o diagnóstico acompanhado de recomendações práticas de mitigação e transferência de conhecimento.
- **Pós-condições:** Relatório de sensibilidade de equipe gerado e disponibilizado para consulta.

Casos de Uso: Validação Estática e Controle de Commits

UC03 - Identificar Arquivos Modificados

- **Identificador:** UC03
- **Ator Principal:** Sistema
- **Objetivo:** Levantar a relação precisa de arquivos prontos para confirmação no controle de versão.
- **Pré-condições:** Execução engatilhada por evento de pré-validação no repositório local.
- **Fluxo Principal:**
    a. O sistema acessa a área de preparação (*staging area*) do repositório Git.
    b. O sistema extrai e compila a lista de todos os artefatos alterados, criados ou renomeados.
    c. O sistema disponibiliza o lote de arquivos para os módulos subsequentes de inspeção.
- **Pós-condições:** Lote de arquivos indexado para auditoria de conteúdo.

UC04 - Analisar Código-Fonte

- **Identificador:** UC04
- **Ator Principal:** Sistema
- **Objetivo:** Examinar as alterações em nível sintático e estrutural.
- **Pré-condições:** Lista de arquivos modificados fornecida com sucesso pelo UC03.
- **Fluxo Principal:**
    a. O sistema itera sobre cada arquivo listado na área de preparação.
    b. O motor aplica conjuntos de regras sintáticas e analíticas para identificar discrepâncias ou padrões suspeitos.
    c. Os achados brutos são estruturados e repassados para a rotina de classificação.
- **Pós-condições:** Extração de padrões concluída.

UC05 - Detectar Problemas no Código

- **Identificador:** UC05
- **Ator Principal:** Sistema
- **Objetivo:** Categorizar desvios identificados na análise estática de acordo com a severidade.
- **Pré-condições:** Dados brutos gerados pela análise do UC04.
- **Fluxo Principal:**
    a. O sistema cruza os padrões identificados com as políticas de qualidade vigentes.
    b. O sistema classifica as ocorrências segundo matriz de risco (ex.: alerta vs. falha crítica).
    c. Os problemas identificados são formalmente registrados no log contextual da sessão.
- **Pós-condições:** Lista consolidada de inconformidades estruturada para subsidiar a tomada de decisão.

UC06 - Bloquear Commit

- **Identificador:** UC06
- **Atores:** Sistema (Principal) e Desenvolvedor
- **Objetivo:** Impedir a persistência de código defeituoso ou vulnerável no histórico local.
- **Pré-condições:** Detecção de inconformidades com criticidade impeditiva no UC05 ou rejeição de formato no UC08.
- **Fluxo Principal:**
    a. O sistema interrompe o gancho de commit em execução (*pre-commit hook*).
    b. O sistema cancela a gravação da transação no repositório Git.

    c. O sistema despacha via terminal/interface uma notificação clara com os motivos do bloqueio e ações necessárias para correção.
- **Pós-condições:** A integridade do branch é mantida sem a inclusão de código inadequado.

UC07 - Permitir Commit

- **Identificador:** UC07
- **Ator Principal:** Sistema
- **Objetivo:** Autorizar a efetivação do commit quando todas as validações forem satisfeitas.
- **Pré-condições:** Ausência de erros impeditivos nas análises estáticas e na padronização da mensagem.
- **Fluxo Principal:**
    a. O sistema consolida as aprovações emitidas pelas etapas anteriores.
    b. O sistema libera o fluxo padrão da ferramenta de versionamento sem interrupções.
- **Pós-condições:** O commit é registrado com êxito no branch de trabalho.

UC08 - Validar Mensagem de Commit

- **Identificador:** UC08
- **Atores:** Sistema (Principal) e Desenvolvedor
- **Objetivo:** Garantir a padronização das descrições textuais associadas aos registros de versionamento.
- **Pré-condições:** Submissão de tentativa de confirmação acompanhada de mensagem.
- **Fluxo Principal:**
    a. O sistema captura a mensagem de commit inserida pelo desenvolvedor.
    b. O sistema avalia a conformidade do texto com o padrão estipulado pela governança (ex.: *Conventional Commits*).
    c. Se a mensagem cumprir as regras, o sistema aprova o formato e prossegue; caso contrário, sinaliza a rejeição do fluxo.
- **Pós-condições:** Validação semântica da mensagem registrada para o ciclo de decisão.

UC09 - Exibir Mensagens ao Usuário

- **Identificador:** UC09
- **Atores:** Sistema (Principal) e Desenvolvedor
- **Objetivo:** Prover feedback claro e contextual sobre o andamento e resultado dos ciclos de validação.
- **Pré-condições:** Conclusão de qualquer etapa de validação, bloqueio ou persistência.

- **Fluxo Principal:**
    a. O sistema compõe mensagens informativas baseadas nos status das validações.
    b. Em caso de sucesso, o sistema renderiza resumos positivos das inspeções.
    c. Em caso de falha, o sistema apresenta detalhadamente a lista de erros e instruções guiadas para resolução.
- **Pós-condições:** Desenvolvedor informado de forma inequívoca sobre o status de sua operação.

Casos de Uso: Ciclo de Vida Pós-Operação

UC10 - Registrar Pós-Commit

- **Identificador:** UC10
- **Ator Principal:** Sistema
- **Objetivo:** Armazenar metadados e métricas derivados de commits concluídos com êxito.
- **Pré-condições:** Confirmação de persistência do commit autorizada pelo UC07.
- **Fluxo Principal:**
    a. O sistema captura os dados contextuais da transação recém-registrada (hash, autor, data/hora e escopo).
    b. O sistema grava os metadados na base de telemetria e atualiza o histórico analítico interno.
- **Pós-condições:** Dados armazenados para composição de métricas de evolução temporal do projeto.

UC11 - Processar Merge

- **Identificador:** UC11
- **Atores:** Sistema e Desenvolvedor
- **Objetivo:** Garantir a reavaliação de conformidade e integridade do repositório após integrações de ramificações.
- **Pré-condições:** Conclusão de operação de fusão (*merge*) entre dois branches.
- **Fluxo Principal:**
    a. O sistema intercepta o evento de união de ramificações.
    b. O sistema aciona rotinas de saneamento e inspeção global do código resultante.
    c. O sistema atualiza o estado operacional e os indicadores consolidados do repositório.
- **Pós-condições:** Repositório atualizado e índices analíticos sincronizados com o branch principal.